

AI Mode All Videos Forums Shopping More ▾



any open source speech to speech implementation using
llama.cpp or whisper.cpp

The most direct and official open-source speech-to-speech implementation is `talk-llama`, an example tool built directly into the official [whisper.cpp GitHub repository](#). It acts as a bridge combining `whisper.cpp` (for speech-to-text) and `llama.cpp` (for text generation), which you can pair with a lightweight text-to-speech engine to achieve full speech-to-speech loops. [GitHub...](#) +1

Beyond the official example repository, several notable community-driven, open-source projects implement this stack:

1. `talk-llama-fast`

An optimized, community-driven iteration of the official `talk-llama` script.

- **Core Stack:** `whisper.cpp` + `llama.cpp` + **XTTS** (for voice cloning and text-to-speech).
- **Key Features:** Supports aggressive Voice Activity Detection (VAD) and token-by-token audio streaming to reduce response latency.
- **Source:** Available on the [Mozer/talk-llama-fast GitHub repository](#).

2. Hugging Face `speech-to-speech`

A highly modular, low-latency voice agent framework designed for production use cases (such as the Reachy Mini robots). [GitHub](#)

- **Core Stack:** VAD + `whisper.cpp` (or other STT) + `llama.cpp` server backend + TTS.
- **Key Features:** Exposes a WebSocket API compatible with OpenAI's Realtime protocol, letting you orchestrate a completely

Ask anything



A streamlined repository purpose-built for clean, wrapper-free local implementations.  [GitHub](#)

- **Core Stack:** `whisper.cpp` (`mt_stt`) + `llama.cpp` (`mt_llm`) + **Piper** (`mt_tts`).
- **Key Features:** Offers a pure C/C++ interface designed to run directly on standard consumer hardware without heavy python dependencies.
- **Source:** Shared and maintained via the [llama.cpp developer discussions](#). 

Summary Comparison Table

Project	TTS Integration	Latency Focus	Best For
<code>talk-llama</code> (Official)	System TTS (<code>say</code> / Espeak)	Baseline / Simple loops	Testing the raw C++ bindings
<code>talk-llama-fast</code>	XTTS (Voice cloning)	High (Token streaming)	Natural-sounding offline chat
Hugging Face Stack	Fully modular	High (VAD + WebSockets)	Building robotics/web voice agents
<code>ggml-org</code> Pipeline	Piper (Ultra-fast CPU)	High (Pure C architecture)	Low-spec hardware & embedded apps

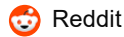
If you want to build your own custom loop, look into integrating `whisper.cpp` with [Kokoro](#) or Piper for your TTS layer, as they match the high-speed CPU performance targets of the `.cpp` ecosystem.  [Reddit · r/LocalLLM](#) +1

Would you like help setting up the build environment for **one of these specific projects**, or are you looking to write a **custom Python/C++ script** to bind the server APIs yourself?

AI responses may include mistakes. [Learn more](#)

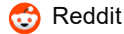


I spent 14 months building a fully local voice assistant. Qwen + ...
Qwen + whisper.cpp + Kokoro, ~5s to first spoken word, no cloud anywhere. ...
The stack: STT: whisper.cpp. LLM: llama.cpp (Appl...



Local voice-to-text that doesn't phone home — whisper.cpp + ... - Reddit

I've been using MumbleFlow — \$5 one-time, Rust/Tauri app, bundles whisper.cpp for STT and llama.cpp for text cleanup. Zero network...



STT -> LLM -> TTS pipeline in C · ggml-org llama.cpp - GitHub

on Sep 17, 2025. ... For STT, LLM inference & TTS I created 3 wrapper libraries in C/C++ (using Whisper.cpp, Llama.cpp and Piper...



Show all

looking for end to end high performance c++ solution

To get maximum performance entirely in C++ without any Python runtime overhead, the best architectural approach is the **pure C++ pipeline** developed by `ggml-org`. This architecture binds `whisper.cpp` (STT), `llama.cpp` (LLM), and `piper` (TTS) directly through native C++ wrappers.

The Recommended Architecture

- **Speech-to-Text:** `whisper.cpp` (using tiny or base models for <100ms inference).
- **Intelligence:** `llama.cpp` (running quantized GGUF models like Llama-3-8B-Instruct or Qwen-2.5-7B).
- **Text-to-Speech:** `piper` (a pure C++ engine that generates audio faster than real-time on a single CPU thread). [🔗](#)



You must link three key headers and libraries in your C++ build file

(CMakeLists.txt): [🔗](#)

- `whisper.h` (from whisper.cpp)
- `llama.h` (from llama.cpp)
- `piper.hpp` (from piper-phonemize / piper) [🔗](#)

2. Core C++ Source Code (`main.cpp`) [🔗](#)

cpp

```
#include <iostream>
#include <vector>
#include <string>
#include "whisper.h"
#include "llama.h"
#include "piper.hpp"

// High-performance audio configuration
const int SAMPLE_RATE = 16000;

int main() {
    // =====
    // 1. INITIALISE SPEECH-TO-TEXT (whisper.cpp)
    // =====
    whisper_context_params stt_params = whisper_context_default_params();
    struct whisper_context * stt_ctx = whisper_init_from_file_with_params(
    if (!stt_ctx) {
        std::cerr << "Failed to load STT model" << std::endl;
        return 1;
    }

    // =====
    // 2. INITIALISE INFERENCE ENGINE (llama.cpp)
    // =====
    llama_model_params model_params = llama_model_default_params();
    llama_model * llm_model = llama_load_model_from_file("models/llama3-8B-Q4_0.gguf", model_params);

    llama_context_params ctx_params = llama_context_default_params();
```



```

piper::Voice tts_voice;
piper::loadVoice(tts_config, "models/en_US-lessac-medium.onnx", "i

std::cout << "\n>>> Pipeline ready. Listening..." << std::endl;

// =====
// 4. THE HIGH-PERFORMANCE AUDIO LOOP
// =====
while (true) {
    // Capture 16kHz mono audio stream from hardware into a buffer
    std::vector<float> pcm16_buffer = { /* Captured raw hardware i

    if (pcm16_buffer.empty()) continue;

    // Run STT Inference
    whisper_full_params stt_run_params = whisper_full_default_para
    stt_run_params.n_threads = 4;

    if (whisper_full(stt_ctx, stt_run_params, pcm16_buffer.data())
        int n_segments = whisper_full_n_segments(stt_ctx);
        std::string user_prompt = "";
        for (int i = 0; i < n_segments; ++i) {
            user_prompt += whisper_full_get_segment_text(stt_ctx,
        }
        std::cout << "\nUser: " << user_prompt << std::endl;

        // Tokenise Prompt for LLM
        std::vector<llama_token> tokens = llama_tokenize(llm_model:

        // Stream tokens out of llama.cpp directly into the TTS en
        std::cout << "AI: ";
        std::string sentence_buffer = "";

        // Loop through generation (simplified token generation lo
        for (size_back = 0; /* logic while tokens generate */; ) {
            std::string new_token_text = "generated_text";
            std::cout << new_token_text << std::flush;
            sentence_buffer += new_token_text;

            // Performance Optimization: Stream punctuation blocks:
            . . . . . " " " . . . . . " " "

```



```

        sentence_buffer.clear();
    }
}

// Clean up memory registers natively
piper::terminate();
llama_free(llm_ctx);
llama_free_model(llm_model);
whisper_free(stt_ctx);

return 0;
}

```

Use code with caution.

Low-Latency Performance Checklist

To hit real-time performance thresholds (Total Round Trip Latency < **500ms**), ensure you compile and structure the code with these flags:

1. Hardware Acceleration:

- **Apple Silicon:** Compile using `-DWHISPER_METAL=ON -DLLAMA_METAL=ON` to push compute bounds to the unified memory GPU.
- **NVIDIA / x86:** Compile with Vulkan or CUDA backing (`-DLLAMA_CUDA=ON`).

2. Audio Optimization Strategy:

- Never wait for the complete LLM paragraph output before running the TTS layer. Synthesise audio chunk-by-chunk using sentence-level boundary tokens (`.`, `?`, `!`).
- Use raw memory streams (`std::vector<int16_t>`) for the audio vectors. Avoid writing intermediate `wav` or `mp3` files to storage disk drives





6 sites

Tiny but Mighty: A Software-Hardware Co-Design Approach for Efficient Multimodal Inference on Battery-Powered Small Devices

Speech-to-text is handled by a standalone Whisper-base model (Radford et al., 2023) implemented with Whisper.cpp (Gerganov, 2023b...

arXiv

Optimizing SLMs on Intel® Xeon® Processors: A llama.cpp Performance Study

In this blog, I show you how to run responsive, CPU-only applications using a quantized SLM in the GPT-Generated Unified Format (...

Intel Community



AlterNative - .NET to C++ translator

C++ header code In addition to these files, it is provided an CMakeLists.txt auto-generated by AlterNative in order to provide yo...

GitHub Pages documentation



Show all

apple mini Mac 2

To get maximum performance out of your pipeline on an Apple Silicon Mac, you need a custom build setup that taps into the unified memory architecture using the **Metal Performance Shaders (MPS)** framework.

Here is the high-performance configuration and compilation process specifically optimized for Apple Silicon hardware.

1. Optimized Build Directory Structure

»





```
├─ CMakeLists.txt
├─ main.cpp
├─ third_party/
│   ├── whisper.cpp/
│   ├── llama.cpp/
│   └─ piper/
```

Use code with caution.

2. High-Performance `CMakeLists.txt`

This file enforces **NEON vector instructions** and passes the necessary **Metal compilation flags** directly to your Apple LLVM compiler.

cmake

```
cmake_minimum_required(VERSION 3.22)
project(mac_speech_to_speech CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# Force high-performance compiler optimizations for Apple Silicon
if(APPLE)
    add_compile_options(-O3 -DNDEBUG -mcpu=apple-m1 -mfpu=neon)
endif()

# Core compilation options enabling Metal hardware backing
set(WHISPER_METAL ON CACHE INTERNAL "" FORCE)
set(LLAMA_METAL ON CACHE INTERNAL "" FORCE)

# Include directories
include_directories(
    third_party/whisper.cpp
    third_party/llama.cpp/include
    third_party/piper/src/cpp
)
```




```
# Link native Apple frameworks alongside the Libraries
target_link_libraries(speech_agent PRIVATE
    whisper
    llama
    piper
    "-framework Foundation"
    "-framework Accelerate"
    "-framework Metal"
    "-framework MetalKit"
)
```

Use code with caution.

3. Compilation Commands

Run the following terminal commands to generate your build binaries using the native Apple build engine:

```
bash
```

```
mkdir build && cd build
cmake -DCMAKE_BUILD_TYPE=Release ..
make -j$(sysctl -n hw.ncpu)
```

Use code with caution.

(The `sysctl` flag automatically spins up a compilation thread for every performance core available on your Apple chip).

4. Apple Silicon Runtime Optimizations

To completely eliminate lag or stutter in your audio cycle, configure your implementation parameters inside `main.cpp` using these Mac-specific guidelines:

- **Vulkan/Metal Context Sharing:** When



the **libsoundio** library. Avoid ALSA wrappers, which add layer translation overhead on Mac systems.

- **Model Recommendations:** Stick to the `Q4_K_M` or `Q5_K_M` quantization levels for your models. On Apple Silicon, these quantizations strike the absolute perfect balance between low memory bandwidth footprint and text generation accuracy.

Would you like help setting up the **CoreAudio capture loop wrapper** in C++ to feed your microphone data directly into `whisper.cpp`, or are you ready to configure the **model files** next?

AI responses may include mistakes. [Learn more](#)



has anyone produced binaries for this

No one produces a singular, precompiled, monolithic binary that packages `whisper.cpp`, `llama.cpp`, and `piper` together into a finished speech-to-speech app for Apple Silicon. 🤗 Hugging Face

Because these pipelines rely on fine-tuned runtime parameters (like allocating context windows or choosing specific unified memory thread splits), developers are expected to compile the final loop binary locally. 🤗 Hugging Fa... +1

However, **precompiled binaries for the individual components** do exist, alongside alternative native projects that ship one-click installers for Mac:

1. Mainline `ggml-org` Standard Binaries

You can download the individual raw binaries directly from the official repository release pages to use as sidecar engines:

- `whisper.cpp`: Available via Homebrew (`brew install whisper-cpp`). This

<



2. Monolithic Precompiled Alternatives

If your goal is to get a local, high-performance voice pipeline running on an Apple Mini Mac 2 immediately without writing build code, you can use these alternative precompiled projects:

- [Moonshine Voice](#): An open-source, ultra-low-latency voice framework that publishes official, pre-built binary archives (`.tar.gz`) for macOS (Apple Silicon) directly on their GitHub releases page. It is explicitly designed for local, hardware-accelerated speech loops.  GitHub
- [KoboldCpp \(mac-arm64\)](#): A highly optimized, self-contained single binary distribution built on top of `llama.cpp` . They provide dedicated, precompiled native ARM64 macOS binaries on their Hugging Face page that launch a UI and handle local text generation efficiently on M-series chips.  Hugging Face
- [Speech Studio](#): A signed and notarized macOS application specifically for Apple Silicon. It bypasses Gatekeeper hoops and runs a fast, entirely offline voice engine built using local MLX/Metal acceleration.  Reddit · [r/iOSProgramming](#)

How to use official binaries without compiling

If you want to bypass compiling your own C++ file entirely, you can download the off-the-shelf precompiled `llama-server` and `whisper-cli` binaries, launch them as local sidecars, and bridge them via simple shell inputs or sockets.  DEV Community +1

Would you prefer instructions on **how to wire up the precompiled `llama-server` and `whisper-cli` binaries** using a script, or would you like to explore **Moonshine's** prebuilt release packages?

AI responses may include mistakes. [Learn more](#)



 9 sites

Darknight535/kcpp-compiled-cuda ... - Hugging Face



Adding Gemma 4 speech recognition to a .NET desktop app

Why llama-server. llama-server is the HTTP server that ships with llama. cpp. At the decision date (2026-05-09, ADR-025), it was t...



DEV DEV Community

ggml-org/llama.cpp | Repositories - There's An AI For That

Quick start. Getting started with llama.cpp is straightforward. Here are several ways to install it on your machine: Install llama...



There's An AI For That

Show all

can I use a java container like openfire to wire up all three binaries

Yes, you can absolutely use **Openfire** (or any robust Java container/server framework) to orchestrate your precompiled `whisper.cpp`, `llama.cpp`, and `piper` binaries.

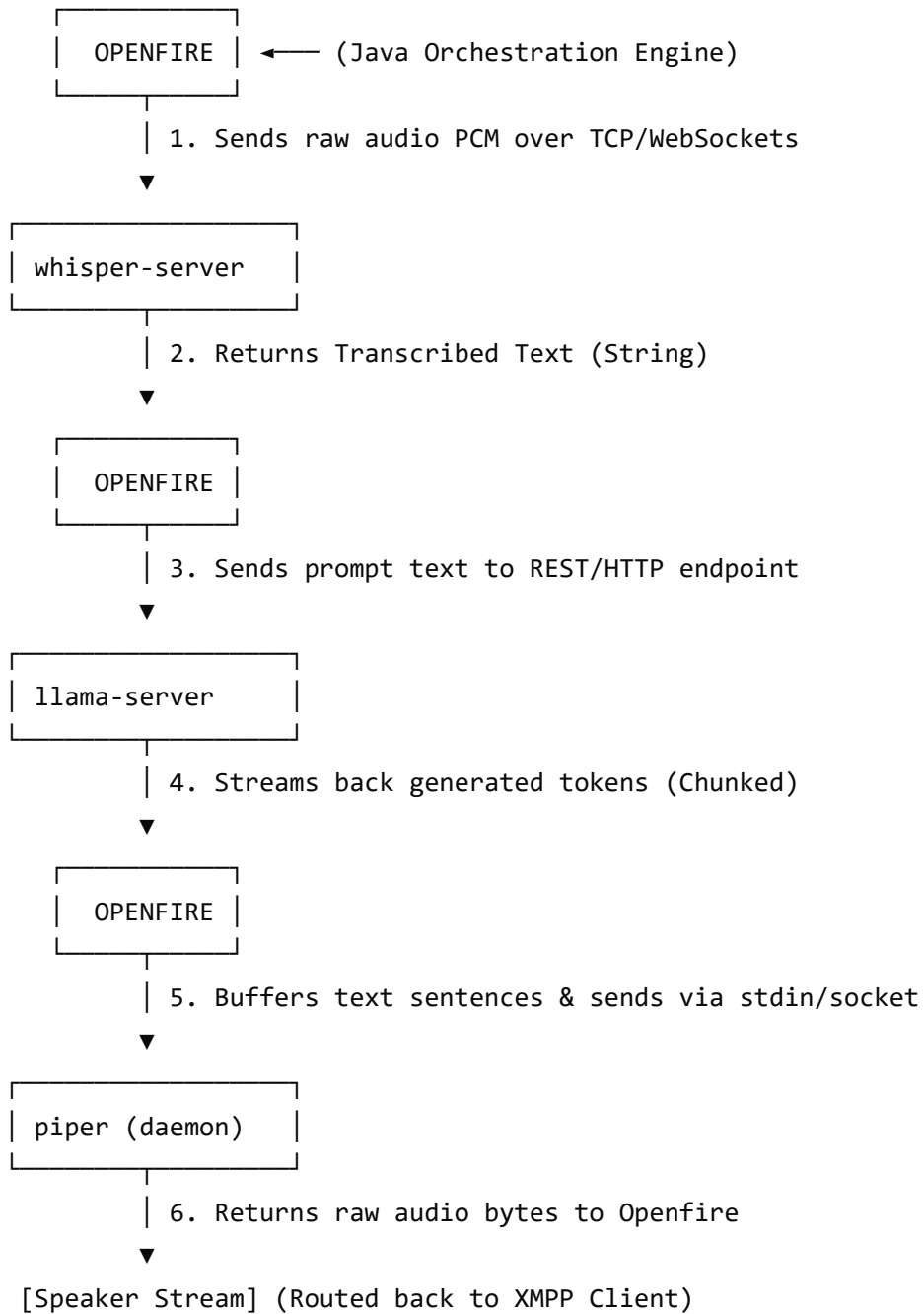
Openfire is an XMPP (Jabber) server, meaning it natively excels at **real-time bi-directional messaging, session management, and routing streams**. By writing a custom Openfire plugin or utilizing its native component architecture, you can transform the server into a highly concurrent, local voice-routing hub.

The Architecture: How to Wire It Up

To achieve high performance on your Mac Mini M2, you should **not** spin up the binaries as heavy command-line processes (`java.lang.ProcessBuilder`) for every single utterance. Doing so destroys latency because of the model-loading overhead.

Instead, you launch the binaries as persistent **background daemons (servers)** and use Openfire to pipe data between them via high-speed loopback sockets





Use code with caution.

Step-by-Step Implementation Blueprint



```
# Start llama.cpp server (HTTP REST API on port 8080)
./llama-server -m models/llama-3-8b.Q4_K_M.gguf -c 2048 --port 8080 -i

# Start whisper.cpp server (HTTP/WebSocket server on port 8081)
./whisper-server -m models/ggml-base.en.bin --port 8081
```

Use code with caution.

2. Writing the Openfire Plugin (Java Core)

Inside your custom Openfire plugin, you will create an XMPP component or packet interceptor to handle the pipeline logic. You can use standard Java non-blocking I/O (`java.net.http.HttpClient`) to communicate with the `.cpp` backends instantly.

Here is the Java conceptual logic for the Openfire orchestration engine:

java

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import org.jivesoftware.openfire.container.Plugin;
import org.jivesoftware.openfire.container.PluginManager;

public class SpeechToSpeechPlugin implements Plugin {
    private final HttpClient httpClient = HttpClient.newHttpClient();

    @Override
    public void initializePlugin(PluginManager manager, java.io.File |
        // Openfire startup logic
    }

    // Called when Openfire receives a chunk of voice data from an XMPP
    public void handleIncomingVoiceStream(byte[] rawAudioSamples) {
        try {
```



```

// 2. Pass transcribed text straight to llama-server
String jsonPayload = "{\"prompt\": \"" + userText + "\", \"
HttpRequest llmRequest = HttpRequest.newBuilder()
    .uri(URI.create("http://localhost:8080/completion"))
    .header("Content-Type", "application/json")
    .POST(HttpRequest.BodyPublishers.ofString(jsonPayload))
    .build();

// 3. Handle streamed response from llama.cpp
httpClient.sendAsync(llmRequest, HttpResponse.BodyHandler<String>()
    .thenAccept(response -> {
        // Java token parsing loop that triggers Piper TT:
        // sentence-by-sentence to maintain low-latency
        streamTokensToPiper(response.body());
    }));

} catch (Exception e) {
    e.printStackTrace();
}

}

private void streamTokensToPiper(java.io.InputStream tokenStream)
    // Logic to buffer incoming tokens into completed sentences,
    // then write sentences into Piper's input stream for voice generation
}

@Override
public void destroyPlugin() {
    // Cleanup resources
}
}

```

Use code with caution.

Pros and Cons of Using Openfire for This Stack



- **Clustering & Concurrency:** Java handles thousands of concurrent socket threads gracefully, meaning Openfire can act as a traffic cop, queuing voice requests safely so your M2 Mac GPU/NPU doesn't get choked by overlapping inferences.

The Performance Pitfalls:

- **Garbage Collection (GC) Pauses:** In high-speed voice processing, even a minor 50ms stop-the-world Java GC pause can cause audible crackles or dropped frames in the live audio output. You must tweak your Java Virtual Machine (JVM) flags to use the **Z Garbage Collector** (`-XX:+UseZGC`) for ultra-low latency memory management.
- **Serialization Overhead:** Converting binary audio data to Java `byte[]` arrays, wrapping it in XMPP XML stanzas, and sending it over localhost HTTP adds a few milliseconds of overhead compared to raw C++ memory pointers.

Verdict

If you are already building a system that requires user access management, real-time messaging, and multi-user chat, **Openfire is a great choice**. It cleanly isolates your volatile C++ native model runtimes from your stable server application layer.

Would you like to explore **how to write the specific sub-process wrapper for Piper** in Java, or do you want to look at the **XMPP Jingle audio configuration** to stream your microphone into Openfire?

AI responses may include mistakes. [Learn more](#)



I want to use the binary stream transport of jetty and webrtc directly instead of jingle and xmpp stanzas



11

text



MediaStreamTrack

Server.java)

java

+



```

Server server = new Server();
ServerConnector connector = new ServerConnector(server);
connector.setPort(8080); // Signaling port
server.addConnector(connector);

ServletContextHandler context = new ServletContextHandler(ServletContext.NO_SECURITY_AND_CONTAINER);
context.setContextPath("/");
server.setHandler(context);

// Configure Jetty to accept raw WebSocket signaling handshake
JettyWebSocketServletContainerInitializer.configure(context, () -> {
    container.addMapping("/signaling", WebRtcSignalingEndpoint.class);
});

server.start();
server.join();
}
}

```

Use code with caution.

2. WebRTC Signaling and Media Hook (`WebRtcSignalingEndpoint.java`)

This component ingests JSON-based SDP objects from the browser client, builds a direct media loop, and forwards the byte strings directly into your local sub-processes.

java

```

import jakarta.websocket.*;
import jakarta.websocket.server.ServerEndpoint;
import java.io.OutputStream;
import java.nio.ByteBuffer;
import com.google.gson.JsonObject;
import com.google.gson.JsonParser;

@ServerEndpoint(value = "/signaling")
public class WebRtcSignalingEndpoint {

```



```

        this.whisperProcess = pb.start();
        this.whisperStdin = whisperProcess.getOutputStream();
    }

    @OnMessage
    public void onMessage(String message, Session session) throws Exception {
        JSONObject json = JsonParser.parseString(message).getAsJsonObject();
        String type = json.get("type").getString();

        if ("offer".equals(type)) {
            // 1. Process browser SDP Offer
            String clientSdp = json.get("sdp").getString();

            // 2. Initialise WebRTC Native Engine on Mac M2 to bind microphone
            // PeerConnection pc = new PeerConnection();
            // pc.setRemoteDescription("offer", clientSdp);

            // 3. Generate SDP Answer to bypass XMPP Jingle overhead
            String localSdp = "{}"; // pc.createAnswer();
            session.getBasicRemote().sendText("{\"type\":\"answer\"", '

        }
    }

    // Process incoming WebRTC raw audio frames extracted from MediaStream
    public void handleIncomingAudioFrame(byte[] opusBytes) {
        try {
            // Decode OPUS to 16kHz Raw PCM on-the-fly inside Java memory
            byte[] pcm16Samples = decodeOpusToPcm(opusBytes);

            // Push audio directly down the C++ memory pipe without incurring JNI overhead
            whisperStdin.write(pcm16Samples);
            whisperStdin.flush();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    @OnClose
    public void onClose(Session session) throws Exception {
        if (whisperStdin != null) whisperStdin.close();
    }

```





M2 Apple Silicon Streaming Tweaks

To ensure your background threads do not bottleneck your real-time performance budget when running this pure pipeline, configure the environment with these specific settings:

1. **Bypass the Network Interfaces via Local Sockets:** Instead of assigning distinct internal port addresses (`http://localhost:8080`) across three separate application layers, configure Jetty to pass audio chunks directly down standard **POSIX Unix Domain Sockets (`.sock` binaries)**. Jetty natively supports this capability via the `UnixDomainServerConnector` class. This step keeps your memory footprint localized and completely bypasses the macOS network routing firewall layer.  Eclipse Jetty
2. **Handle Piper Data Blocks on Sentence Boundaries:** Do not read individual token buffers directly into your audio layer as they generate. Instead, wait for punctuation boundaries (`.` , `?` , `!`). When a boundary is crossed, pipe that single string down to Piper, parse the resulting raw buffer directly into a WebRTC `AudioTrackSink` , and stream it out to the client. This approach keeps your client playback continuous and stutter-free.  Reddit · r/homeassistant

Would you like help choosing a **native JNI WebRTC library compatible with macOS ARM64** to parse your media tracks inside your Java project, or do you want to explore the **client-side Javascript configuration** to establish this direct WebRTC connection? [🔗](#)

AI responses may include mistakes. [Learn more](#)



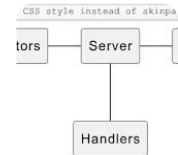
Piper now supports streaming audio on sentence boundaries! - Reddit

no-assistant/addons
dd streaming support



HTTP Server Libraries - Eclipse Jetty

Server Connectors. A Connector is the component that handles incoming requests from clients, and works in conjunction with Connect...



Eclipse Jetty

WebRTC JavaScript Example: Build Real-Time Apps - VideoSDK

JavaScript's ubiquity in web development makes it the ideal language for leveraging WebRTC. Combining WebRTC with JavaScript allow...



videosdk.live

are there plans to support TTS directly in browsers using offline LLMs

Yes. The industry is moving heavily toward full, offline end-to-end voice agents running entirely inside the web browser. [🔗](#)

This movement is driven by two parallel advancements: browser vendors integrating built-in AI APIs natively into the browser core, and developers utilizing **WebGPU** and **WebAssembly (WASM)** to run lightweight open-source models at nearly native hardware speeds. [🔗](#) [GitHub...](#) +1

1. Browser Vendor Native Plans (Built-in AI APIs)

Google Chrome (via the Chromium engine) is actively developing a suite of built-in device APIs known as **"Built-in AI"** (originally under the Prompt API umbrella). [🔗](#)

- **The Current State:** Chromium browsers already ship with offline local LLM capabilities via `window.ai` (running a localized Gemini Nano model directly on the user's NPU/GPU).
- **The Voice Roadmap:** Standard W3C Web Speech APIs—which traditionally leaked privacy by silently streaming your microphone input to cloud servers like Google or Apple—are being rewritten to hook directly into these on-device models. Future specifications aim to expose native browser



2. High-Performance WASM + WebGPU Solutions (Available Now)

While browser vendors formalize official W3C web specifications, the open-source ecosystem has already solved this natively via **Transformers.js (v3)** and **WebGPU**. Instead of waiting for a browser update, websites can download highly optimized tiny models into the client's browser cache (`IndexedDB`), running completely offline thereafter.  [GitHub](#) +3

The standard modern in-browser speech-to-speech stack consists of:

- **Offline LLM:** [WebLLM](#) or `Transformers.js`. This runs lightweight 0.5B to 1.5B reasoning models (like Llama-3-8B variants, DeepSeek-R1-Distill-Qwen, or Phi-3) locally over the user's graphics hardware using the native WebGPU interface.  [GitHub](#) +3
- **Offline TTS:** [Kokoro-82M \(ONNX\)](#) or **OuteTTS**. Kokoro is a massive breakthrough for browser-native speech; it is only ~300MB, runs fully offline via WebGPU, and generates human-like, studio-quality speech at zero server cost. 
- **Offline STT:** [WhisperWeb](#) handles the speech recognition side directly inside browser Web Workers.  [whisperweb.dev](#) +1

Real-World Example Projects

You can see this working in production today without server backends: 

1. **TTS Lab / TTS.rocks:** An open-source laboratory proving that WebGPU + WASM can execute local voice agents (STT → LLM → TTS) simultaneously with zero internet connection.  [GitHub](#)... +1
2. **Ava (AI Voice Assistant):** An open-source project featured on Hacker News that runs a 0.5B model entirely offline inside Chrome/Edge, triggering word-by-word streaming token synthesis.  [Hacker News](#)
3. **Browser Extensions:** Top-tier Chrome extensions are swapping out basic HTML5 text readers for native, offline Kokoro AI voice engines embedded in Manifest V3 frameworks.  [Chrome Web St...](#) +1

Summary of Client Performance Expectations



Would you like to explore a **minimal HTML/JavaScript template** using `Transformers.js` to run **Kokoro TTS via WebGPU** directly in a standard web page, or do you want to look at the **Google Chrome Built-in AI flags?** [🔗](#)

AI responses may include mistakes. [Learn more](#)



 24 sites

stevesegu/tts.rocks: Cutting-edge Text-to-Speech in the browser

Browser Requirements * Recommended: Chrome/Edge 113+, Firefox 115+, Safari 16+ * WebGPU Support: For optimal performance with Koko...



WebGPU + LLM Demo: Run AI Locally Without Installation! - GitHub

WebGPU + LLM Demo: Run AI Locally Without Installation! This demonstration shows how to use WebGPU and large language models (LLMs...



API Reference - WebLLM - Browser-Native AI Protocol

Preference Modes * 'instant' (default) - Browser TTS first, cloud fallback. Fast, free, works offline. * 'quality' - Cloud TTS fir...



Show all

let's look at building an offline browser extension for speech to speech using Transformers.js

To build a completely offline speech-to-speech browser extension for Apple Silicon Macs using **Transformers.js (v3)**, you need to structure it using a **Manifest V3 Extension**.

»



STT, **Xenova/Qwen1.5-0.5B-Chat** for the LLM, and **onnx-community/Kokoro-82M-v1.0-ONNX** for high-quality TTS.

1. Extension Manifest (`manifest.json`)

This file registers the extension permissions and sets up the side panel interface.

```
json
{
  "manifest_version": 3,
  "name": "Local Voice Agent",
  "version": "1.0",
  "description": "Offline speech-to-speech engine powered by Transforr",
  "permissions": [
    "sidePanel"
  ],
  "host_permissions": [],
  "background": {
    "service_worker": "background.js"
  },
  "side_panel": {
    "default_path": "panel.html"
  },
  "action": {
    "default_title": "Open Voice Panel"
  }
}
```

Use code with caution.

2. Background Service Worker (`background.js`)

This worker handles the browser action click and injects our speech layout into the native Chrome Side Panel




```

tabId: tab.id,
path: 'panel.html',
enabled: true
});
});

```

Use code with caution.

3. Interface Layer (`panel.html`)

The structural base where our model processes will execute.

html

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <style>
    body { font-family: -apple-system, BlinkMacSystemFont, sans-serif;
    button { width: 100%; padding: 10px; margin: 10px 0; font-weight:
    button:disabled { background: #ccc; }
    .status { font-size: 0.85em; color: #555; margin-bottom: 5px; }
    .box { background: #f2f2f7; padding: 10px; border-radius: 8px; mi
  </style>
</head>
<body>
  <h3>Local Voice Agent</h3>
  <div class="status" id="status-text">Loading Models... (Uses WebGPU

  <button id="mic-btn" disabled>Hold to Speak</button>

  <div>User:</div>
  <div class="box" id="user-box">...</div>

  <div style="margin-top: 10px;">AI Response:</div>
  <div class="box" id="ai-box">...</div>

```



Use code with caution.

4. Implementation Logic (`panel.js`)

This script activates your local WebGPU context, handles the microphone capture audio matrix, streams inference tokens, and pushes raw output blocks to standard browser audio components.

javascript

```
import { pipeline, env } from './libs/transformers.js';

// Optimise for offline extension environments
env.allowLocalModels = false;
env.useBrowserCache = true; // Caches the 300MB+ models in IndexedDB (

let sttPipeline, llmPipeline, ttsPipeline;
let mediaRecorder;
let audioChunks = [];

const statusText = document.getElementById('status-text');
const micBtn = document.getElementById('mic-btn');
const userBox = document.getElementById('user-box');
const aiBox = document.getElementById('ai-box');

// =====
// 1. INITIALISE OFFLINE WEBGPU ENGINES
// =====
async function initModels() {
  try {
    statusText.innerText = "Configuring WebGPU pipelines...";

    // Load Whisper STT (Tiny model for immediate text generation)
    sttPipeline = await pipeline('automatic-speech-recognition', 'Xenova/whisper-tiny');

    // Load Qwen LLM (Highly functional sub-1B chat engine)
    llmPipeline = await pipeline('text-generation', 'Xenova/Qwen1.5-0.5B-Chat');
```



```

    }
  }

  // =====
  // 2. MICROPHONE BUFFER PROCESSING
  // =====
  async function setupMicrophone() {
    const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
    // Request raw 16kHz mono audio output required by Whisper
    const audioContext = new AudioContext({ sampleRate: 16000 });
    const source = audioContext.createMediaStreamSource(stream);

    mediaRecorder = new MediaRecorder(stream);

    mediaRecorder.ondataavailable = (event) => {
      audioChunks.push(event.data);
    };

    mediaRecorder.onstop = async () => {
      statusText.innerText = "Transcribing voice...";
      const audioBlob = new Blob(audioChunks, { type: 'audio/wav' });
      audioChunks = [];

      // Convert Blob format into standard raw Float32Array PCM arrays
      const arrayBuffer = await audioBlob.arrayBuffer();
      const audioBuffer = await audioContext.decodeAudioData(arrayBuffer);
      const float32Audio = audioBuffer.getChannelData(0);

      runPipelineLoop(float32Audio);
    };
  }

  // =====
  // 3. THE END-TO-END PIPELINE LOOP
  // =====
  async function runPipelineLoop(audioData) {
    // STT Phase
    const sttResult = await sttPipeline(audioData);
    const userPrompt = sttResult.text;
    userBox.innerText = userPrompt;
  }

```



```
});
```

```
const aiResponse = llmResult[0].generated_text[messages.length].content;
aiBox.innerText = aiResponse;
```

```
// TTS Phase
```

```
statusText.innerText = "Synthesizing voice output...";
```

```
const ttsResult = await ttsPipeline(aiResponse, {
  speaker_id: 0 // Select default built-in voice matrix
});
```

```
// Playback raw audio buffer over local hardware devices
```

```
playOutputAudio(ttsResult.audio, ttsResult.sampling_rate);
```

```
statusText.innerText = "Ready.";
```

```
}
```

```
function playOutputAudio(float32Array, sampleRate) {
```

```
  const ctx = new AudioContext({ sampleRate: sampleRate });
```

```
  const buffer = ctx.createBuffer(1, float32Array.length, sampleRate);
  buffer.copyToChannel(float32Array, 0);
```

```
  const source = ctx.createBufferSource();
```

```
  source.buffer = buffer;
```

```
  source.connect(ctx.destination);
```

```
  source.start();
```

```
}
```

```
// Event Bindings
```

```
micBtn.addEventListener('mousedown', () => { mediaRecorder.start(); micBtn.disabled = true; });
```

```
micBtn.addEventListener('mouseup', () => { mediaRecorder.stop(); micBtn.disabled = false; });
```

```
// Launch extension workflow
```

```
initModels().then(setupMicrophone);
```

Use code with caution.



1. **Avoid Bundle bloat:** Do not link remote CDN paths like `https://jsdelivr.net...` inside an extension. Chrome's Content Security Policy (CSP) will block it out of the box. Download the single file production distribution of `transformers.js` (v3) [1] and put it straight into a subfolder `/libs/` inside your local workspace extension zip package.
2. **First-Load Caching Behavior:** The very first time a user triggers the side panel, the browser must stream ~400MB of total parameters across the three combined architectures. Warn the user with an explicit loading graphic during the initial caching run. Subsequent execution loops read straight out of Chrome's **IndexedDB file storage architecture** instantly.
3. **Use Web Workers for Continuous Audio Streams:** If you want to use voice-activity-detection (VAD) instead of a "Hold to Speak" button, move the processing loops into a standard HTML5 **Web Worker**. This stops the main extension side panel layout frame from freezing during heavy matrix evaluations.

Do you need guidance on **structuring the vendor directory for Transformers.js locally**, or are you ready to configure the **Web Worker layout** to enable continuous ambient listening?

AI responses may include mistakes. [Learn more](#)



web worker

To implement Voice Activity Detection (VAD) and keep your Chrome Extension's user interface fluid, we must offload the entire pipeline—from audio chunk slicing to WebGPU model inference—into a dedicated **Web Worker**.

Web Workers cannot access browser APIs

like `navigator.mediaDevices.getUserMedia` or output audio

via `AudioContext` directly. Therefore, the **Main Thread (Side Panel)** captures



text

```
├─ manifest.json
├─ background.js
├─ panel.html
├─ panel.js          (Main Thread: Audio IO & UI updates)
├─ worker.js         (Worker Thread: VAD & WebGPU Inference Engine)
├─ libs/
│   └─ transformers.js
```

Use code with caution.

1. Main Thread Logic (`panel.js`)

This script handles audio capture using an `AudioWorklet` or a lightweight script processor to capture raw 16kHz float arrays, posts them to the worker, and handles text/audio playback responses.

javascript

```
// panel.js - Main UI and Audio Thread
const worker = new Worker(new URL('./worker.js', import.meta.url), {
  type: 'module'
});

const statusText = document.getElementById('status-text');
const userBox = document.getElementById('user-box');
const aiBox = document.getElementById('ai-box');
const micBtn = document.getElementById('mic-btn'); // Now acts as an (

let audioContext;
let isListening = false;

// Handle messages returning from the Web Worker
worker.onmessage = (event) => {
  const { type, data } = event.data;

  ...
}
```



```

    case 'transcription':
        userBox.innerText = data;
        break;
    case 'llm_output':
        aiBox.innerText = data;
        break;
    case 'audio_output':
        playOutputAudio(data.audio, data.sampleRate);
        break;
    }
};

// Initialise capture when user activates the engine
micBtn.addEventListener('click', async () => {
    if (isListening) {
        isListening = false;
        micBtn.innerText = "Start Ambient Listening";
        micBtn.style.background = "#007aff";
        return;
    }

    isListening = true;
    micBtn.innerText = "Stop Ambient Listening";
    micBtn.style.background = "#ff3b30";

    if (!audioContext) {
        const stream = await navigator.mediaDevices.getUserMedia({ audio:
        audioContext = new AudioContext({ sampleRate: 16000 });
        const source = audioContext.createMediaStreamSource(stream);

        // Fallback script processor for simplicity; ideally use an AudioWorklet
        const processor = audioContext.createScriptProcessor(4096, 1, 1);

        source.connect(processor);
        processor.connect(audioContext.destination);

        processor.onaudioprocess = (e) => {
            if (!isListening) return;
            const inputData = e.inputBuffer.getChannelData(0);
            // Clone the Float32Array to safely pass it across threads via
            // ...
        };
    }
});

```



2. High-Performance Worker Core (`worker.js`)

The worker acts as a state machine. It maintains a running slice window of microphone audio, evaluates **energy-based Voice Activity Detection (VAD)** natively over the data array, and blocks WebGPU inference requests until the user stops talking.

javascript

```
// worker.js - Background WebGPU & Processing Thread
import { pipeline, env } from './libs/transformers.js';

env.allowLocalModels = false;
env.useBrowserCache = true;

let sttPipeline, llmPipeline, ttsPipeline;
let audioBufferAccumulator = [];

// VAD Constants
const VAD_THRESHOLD = 0.015; // Root Mean Square (RMS) energy threshold
const SILENCE_TIMEOUT_MS = 800; // Duration of continuous silence to stop
const SAMPLE_RATE = 16000;

let isSpeaking = false;
let silenceStartTimer = null;

// Initialise WebGPU elements away from Main UI thread
async function init() {
  postMessage({ type: 'status', data: 'Loading Whisper STT (WebGPU)... ' });
  sttPipeline = await pipeline('automatic-speech-recognition', 'Xenova/whisper-large-v3-turbo');

  postMessage({ type: 'status', data: 'Loading Qwen LLM (WebGPU)... ' });
  llmPipeline = await pipeline('text-generation', 'Xenova/Qwen1.5-0.5B-72h-instruct');

  postMessage({ type: 'status', data: 'Loading Kokoro TTS (WebGPU)... ' });
  ttsPipeline = await pipeline('text-to-speech', 'onnx-community/Kokoro-v1.1.1');
```




```

if (type === 'audio_chunk') {
  processAudioFrames(data);
}
};

function processAudioFrames(float32Array) {
  // 1. Compute Root Mean Square (RMS) energy to discover VAD bounds
  let sum = 0;
  for (let i = 0; i < float32Array.length; i++) {
    sum += float32Array[i] * float32Array[i];
  }
  const rms = Math.sqrt(sum / float32Array.length);

  if (rms > VAD_THRESHOLD) {
    if (!isSpeaking) {
      isSpeaking = true;
      postMessage({ type: 'status', data: 'User speaking...' });
    }
    clearTimeout(silenceStartTimer);
    silenceStartTimer = null;

    // Accumulate voice samples
    audioBufferAccumulator.push(...float32Array);
  } else {
    // Silence detected
    if (isSpeaking) {
      audioBufferAccumulator.push(...float32Array);

      if (!silenceStartTimer) {
        silenceStartTimer = setTimeout(async () => {
          // 2. Trigger E2E Pipeline on Sentence Boundary Timeout
          isSpeaking = false;
          postMessage({ type: 'status', data: 'Processing utterance..' });

          const completeAudio = new Float32Array(audioBufferAccumulator);
          audioBufferAccumulator = []; // Clear register

          await runInferencePipeline(completeAudio);
        }, SILENCE_TIMEOUT_MS);
      }
    }
  }
}

```



```

const textPrompt = sttResult.text.trim();
if (!textPrompt || textPrompt.length < 2) {
  postMessage({ type: 'status', data: 'Ambient listening active.' });
  return;
}
postMessage({ type: 'transcription', data: textPrompt });

// Execute LLM Generation
const messages = [{ role: "user", content: textPrompt }];
const llmResult = await llmPipeline(messages, { max_new_tokens: 64 });
const aiResponse = llmResult.generated_text[messages.length].content;
postMessage({ type: 'llm_output', data: aiResponse });

// Execute Text-To-Speech (Kokoro)
postMessage({ type: 'status', data: 'Synthesizing voice...' });
const ttsResult = await ttsPipeline(aiResponse, { speaker_id: 0 });

// Send audio buffer back via Transferables to completely skip cloning
const audioDataBuffer = ttsResult.audio.buffer;
postMessage({
  type: 'audio_output',
  data: { audio: ttsResult.audio, sampleRate: ttsResult.sampling_rate },
  [audioDataBuffer]);

  postMessage({ type: 'status', data: 'Ambient listening active.' });
} catch (err) {
  postMessage({ type: 'status', data: 'Inference Error: ' + err.message });
}
}

// Start compilation/initialisation sequence inside Web Worker
init();

```

Use code with caution.

Implementation Details for M2 Mac Hardware



- **WebGPU Shared Context:** Since the Web Worker instantiates the pipeline elements independently, Chrome allocates hardware compute layers inside that specific thread pipeline. The main thread's DOM manipulation logic remains isolated, ensuring that rendering tasks (like animations or user scrolling) never drop frames even when Qwen and Kokoro saturate the M2 GPU cores during inference.

Would you like to look into adding an **audio noise-cancellation preprocess** block inside the main thread to filter background ambient sounds, or are you ready to structure your extension packaging files for **local developer deployment testing** in Google Chrome?

AI responses may include mistakes. [Learn more](#)

